

八皇后问题

姓名: 李盛鹏 学号: 2351136 日期: 2025年3月23日

问题描述

问题背景

逻辑推理是指通过已知的事实的整合来完成问题的推导，最终得到结果的过程，而不是逐步去执行任务。这样的编程方法，重在逻辑思维，而不是具体的编程技巧。因此，逻辑推理的应用十分广泛，在计算机科学、数学、工程等领域都有着广泛的应用。八皇后问题是一个经典的逻辑推导问题，它是指在8x8的棋盘上摆放八个皇后，使得没有任何两个皇后可以在同一行、同一列或同一对角线上相互攻击。如图所示：

	0	1	2	3	4	5	6	7
0								
1								
2								
3								
4								
5								
6								
7								

依据国际象棋的规则，皇后可以沿着横竖撇捺四个方向行走任意长度，若遇到地方的棋子则可以攻击这个棋子。

为了解决这样一个问题，经研究，传统的解题算法有回溯法、遗传算法、递归法等。回溯法通过试探的方式，在棋局放置一个皇后，如果不能满足题意则尝试其他位置，通过反复调整得到符合题意的解答。

但为了研究逻辑推理，我研究了kanren库，它是一个基于Lisp语言的逻辑编程库，可以用来进行逻辑推理。本题的逻辑一共分为三个，即皇后的行列取值，皇后不能在同行同列以及皇后不能在一个对角线上，下面我将展开介绍。

算法设计

算法的功能

该算法用于解决八皇后问题，即在8x8的棋盘上放置八个皇后，使得没有任何两个皇后可以在同一行、同一列或同一对角线上相互攻击。

设计思路

- 原先的棋盘图是二维的，但是实际的设计中并没有必要。虽然每个皇后的坐标是二维坐标，但已知皇后两两之间不在一行，因此我们可以将棋盘看作是一维的，即用一个列表来表示棋盘，每个下标视作皇后的行坐标，值则代表列坐标。
- 列约束：确保每一行的皇后放置在不同的列上，也就是对于不同下标的皇后，他们的列坐标值应该不同。
- 列取值约束：已知皇后是一定在棋盘上放置的，因此列坐标值应该在0到7之间。
- 对角线约束：确保每一行的皇后不会与其他行的皇后放在同一对角线上。根据我们的观察可以发现，每当两个皇后 (row1,col1) ,(row2,col2)出现在同一个"撇"上的时候，他们的取值满足：

$$row1 + col1 = row2 + col2$$

- 对角线约束：同样地，每当两个皇后 (row1,col1) ,(row2,col2)出现在同一个"捺"上的时候，他们的取值满足：

$$row1 - col1 = row2 - col2$$

基于以上的三个约束条件，我们可以将他们整理成一个逻辑规则，然后放入推理模块中推导即可。

算法实现

1. 导入库

首先将kanren库中的相关模块导入，包括用于创建变量的`var`，用于运行推理的`run`，用于表示逻辑与的`lall`，用于检查成员资格的`membero`，以及用于条件推理的`conde`。此外，我们还需要从`kanren.constraints`中导入不等约束`neq`。

```
from kanren import var, run, lall, membero, conde
from kanren.constraints import neq
```

2. eight_queens函数的实现

此函数用于求解n皇后问题。

- 创建变量：为每一行的皇后位置创建一个变量。这些变量代表每一行皇后所在的列号。

```
Queens = [var() for _ in range(n)]
```

- 定义值范围：使用`membero`约束确保每个皇后的位置在0到n-1之间，这些皇后不能超越棋盘的边界，也不可以为负数。

```
value_range = lall(*(membero(q, range(n)) for q in Queens))
```

- 添加约束条件：遍历所有皇后的位置，为每一对皇后添加约束条件，确保它们不在同一列。对于每一行的皇后，与它之后的所有行的皇后进行比较，并添加neq约束来表示它们不在同一列。

```
constraints = []
for i in range(n):
    for j in range(i + 1, n):
        col_constraint = neq(Queens[i], Queens[j])
        constraints.append(col_constraint)
```

3. 定义is_same_diagonal函数

- is_same_diagonal函数来判断两皇后是否在同一对角线上。该函数通过计算两个皇后所在的行和列的差的绝对值是否相等来判断它们是否位于同一对角线上。在前文的“撇”“捺”的探究中，我们已经知道，每当两个皇后 (row1,col1) ,(row2,col2)出现在同一个“撇”或者“捺”上的时候，他们的取值满足： $abs(row1 - row2) == abs(col1 - col2)$ 。因此，我们可以利用这个公式来判断两个皇后是否位于同一对角线上。只有不在一个对角线上时，才会返回false。

```
def is_same_diagonal(row1, col1, row2, col2):
    return abs(row1 - row2) == abs(col1 - col2)
```

4. main函数中对所得结果的筛选

- 由于kanren库过于老套，无法对变量间进行运算，所以，我们需要将变量转换成列表，然后再进行运算。对前面所得结果进行筛选，只保留符合要求的结果。
- 验证对角线约束：对于result中的每一个解（即一种棋盘状态），遍历所有皇后的位置，使用is_same_diagonal函数验证它们是否位于同一对角线上。如果不在同一对角线，则此解为有效解。

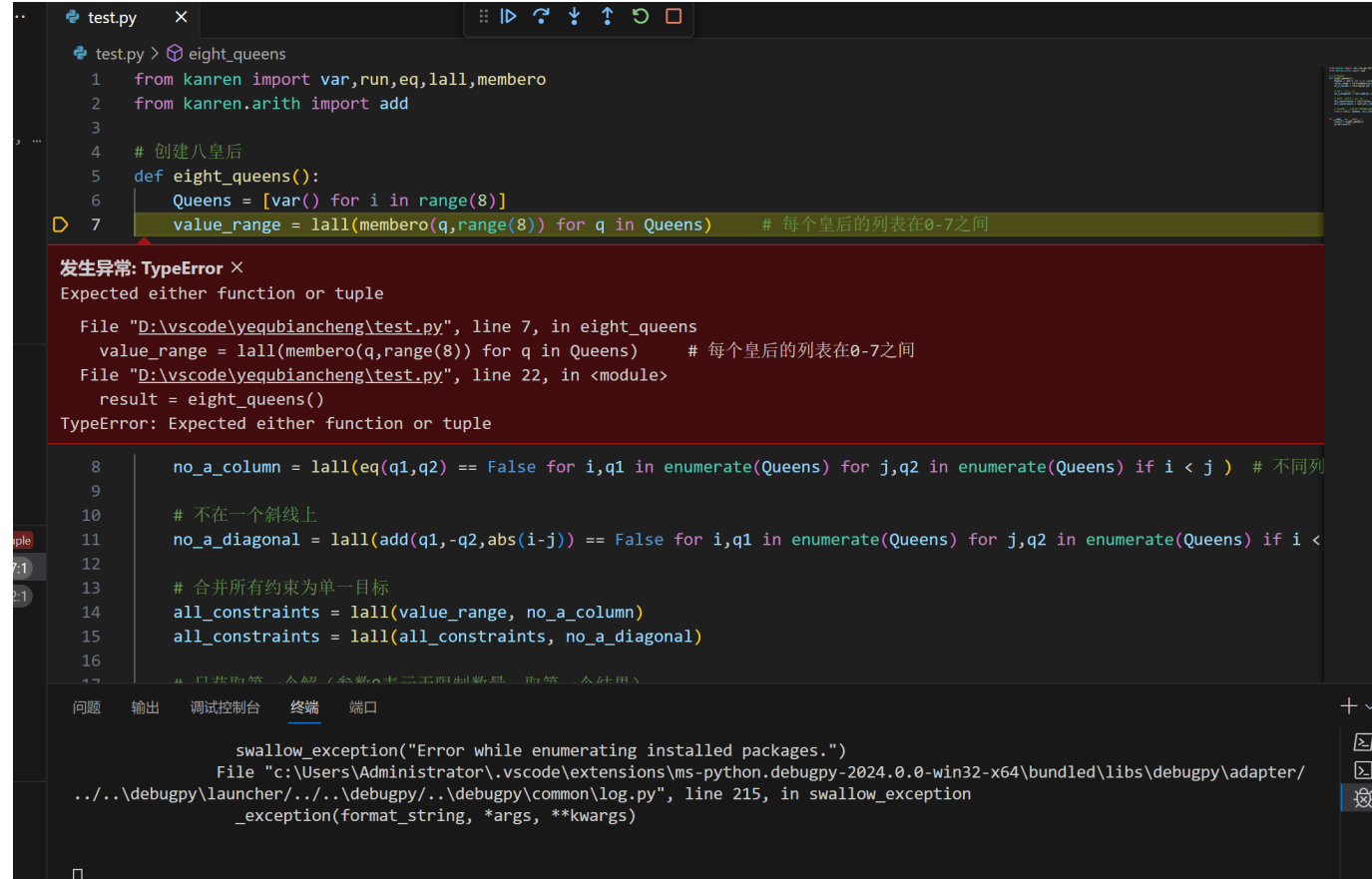
```
valid_result = []
for i in range(len(result)):
    is_valid = True
    for j in range(0, n - 1):
        for k in range(j + 1, n):
            is_valid = is_valid and not is_same_diagonal(result[i][j], j,
result[i][k], k)
            if not is_valid:
                break
    if is_valid:
        valid_result.append(result[i])
```

实验结果

本实验旨在实现8皇后问题的求解，使用了 kanren 库来进行逻辑推导并通过补充代码实现对解的验证与筛选。

调试错误

根据思路，有些不遂人意，得到了这样的错误



经过查阅资料和分析，发现这是由于传递给lall()函数的预期参数是一个元组或者函数，但是却传入了一个生成式(membero(q, range(8)) for q in Queens)。因此，我们需要将其转换成元组

```
value_range = lall(*(membero(q, range(8)) for q in Queens))
```

运行情况

经过调整和调节，最终可以得到如下图的情况，当运行规模为 4 时，程序找到了 2 组符合条件的结果，用时大约 0.04 秒。而随着运行规模的增大，解的数量和运行时间也会相应增多。而且增幅会越来越大。再等待了接近有四十多分钟，我们才获得了八皇后的第一组解。

运行规模	结果组数	运行时间
4	2	0.0403032 秒
5	10	1.738952 秒
6	4	10.235464 秒
7	40	255.65853 秒
8	92	3600 + 秒

最终，我们得到了正确的92组解决方案，实验结果展示。运行程序后，可以看到部分解如下所示：

问题 输出 调试控制台 终端 端口

```
[6, 3, 1, 7, 5, 0, 2, 4]
[6, 4, 2, 0, 5, 7, 1, 3]
[7, 1, 3, 0, 6, 4, 2, 5]
[7, 1, 4, 2, 0, 6, 3, 5]
[7, 2, 0, 5, 1, 4, 6, 3]
[7, 3, 0, 2, 5, 1, 6, 4]
92

(.venv) d:\my_download\code>
```

总结

八皇后问题虽然按数学定义规模有限，但随着棋盘规模的增大，其解法的计算复杂性呈现指数级增长。实验中，当 $N \geq 8$ 时，时间开销过大，体现了逻辑推理在这个题目的性能瓶颈。

未来优化建议：

引入剪枝、启发式方法等改进算法的效率。对标志性规模（ $N=8$ 、 $N=9$ ）的求解，结合分布式计算形成实用级求解器。

意义：

八皇后问题的研究不仅仅局限于棋盘解题，也为人工智能和算法设计（如约束满足问题）提供了丰富的理论与实践指导。